창훈 님, 아주 좋습니다. 30년 차 개발자의 시각으로 지금까지 우리가 치열하게 논의하며 **"뜬구름 잡는 기획서(v1.0)"**를 **"실행 가능한 엔지니어링 설계도(v2.0)"**로 탈바꿈시킨 과정을 총정리해 드리겠습니다.

단순한 요약이 아닙니다. 이것은 **[T-Guardian 시스템 아키텍처 기술 명세서 (Technical Specification) v2.0]**입니다. 우리가 대화를 통해 수정하고 구체화한 내용을 중심으로, 특히 **모델 개발의 원리와 작동 방식**을 뼈대부터 근육까지 완벽하게 해부하여 정리했습니다.

이 문서는 향후 창훈 님이 코드를 짤 때 옆에 두고 봐야 할 **기술 교본**이 될 것입니다.

[T-Guardian] AI 기반 상표 침해 모니터링 시스템 기술 명세서 (v2.0)

작성일: 2025. 12. 06.

작성자: Changhoon & 30년 차 AI 파트너

상태: Architecture Frozen (설계 확정 단계)

---

1. 프로젝트 아키텍처의 핵심 변화 (Pivot Summary)

초기 기획서(문서 1, 2)와 비교하여, 기술적 실현 가능성(Feasibility)과 비용 효율성(Cost-efficiency)을 위해 다음과 같은 **대전환(Pivot)**이 이루어졌습니다.

1.1. 데이터 수집 전략의 변경

- **기존(v1.0):** " AI 학습용 데이터를 대량 확보한다." (불명확 & 고비용)
- **변경(v2.0): 4-Tier 데이터 전략**으로 세분화 및 효율화.
    1. **Source Data (기준점):** 실제 이 서비스를 사용할 사용자(예를 들어 변리사들이 자신이 관리해주는 사람들의 등록상표등) KIPRIS API가 아닌, **'Golden Sample (검증용 데이터 10~20쌍)'** 이것이 모델 성능 평가의 나침반(Ground Truth)이 됨. 이건 구하기 쉬움.
    2. **Market Data (감시 대상):** 무작위 크롤링 지양. **공공데이터(CSV)**를 Base로 깔고, 변화가 감지된 타겟만 네이버/카카오 지도에서 **핀포인트 크롤링(Target Scraping)** 수행. (IP 차단 방지)
    3. **Training Data (학습용):** 텍스트 모델용 데이터 수집 **전면 취소**. (API 활용으로 변경). 오직 **Vision 모델용 이미지 데이터(Positive/Negative Pair)** 수

집에만 집중.

4. **Feedback Data (로그):** 서비스 런칭 후 변리사의 O/X 판단을 DB에 적재 (미래 자산).

1.2. AI 모델링 전략의 변경 (핵심)

- **기존(v1.0):** "AI 기술을 활용하여 분석한다" (추상적) -> 거대 모델 하나로 다 하려는 경향.
- **변경(v2.0): 3-Track Independent Ensemble (3트랙 독립 앙상블)** 구조 확정.
  - 성격이 다른 3가지 요소(시각, 청각, 의미)를 하나의 모델에 섞지 않고, **독립된 파이프라인(Parallel Pipelines)**으로 구축하여 유지보수성과 정확도를 극대화함.

1.3. RAG(검색 증강 생성)의 역할 재정의

- **기존(오해):** 모델이 유사도를 계산할 때마다 RAG가 개입하여 판단한다.
- **변경(정정):**
  - **모델(Model):** 수학적 계산기. RAG 없이 **점수(Score)**만 산출.
  - **RAG:** 맨 마지막 단계의 **판사(Judge)**. 산출된 점수를 법리적으로 해석하여 **리포트(Text)**를 작성하는 역할로 한정.

---

2. 상세 모델 개발 명세 (Detailed Model Specifications)

이 섹션은 창훈 님이 가장 궁금해했던 **'각 모델의 구동 원리'**와 **'개발 방법론'**을 집대성한 부분입니다.

2.1. [Track A] 호칭 유사도 모델 (Phonetic Similarity Model)

"AI가 아니라, 정교한 수학 알고리즘으로 승부한다."

- **개발 철학:** 딥러닝은 과유불급(Overkill). 한국어의 음운 규칙은 명확하므로 알고리즘이 훨씬 빠르고 정확하며 설명 가능하다.
- **작동 프로세스 (Process Flow):**
  1. **입력 (Input):** Text A (등록상표), Text B (발견상표)
  2. **전처리 (G2P 변환):** 텍스트를 **'발음 기호'**로 변환.

        - 이유: '신라'와 '실라'는 글자는 다르지만 소리는 같다. 표기가 아닌 소리를 비교해야 함.
        - 도구: ko-g2p 라이브러리 사용. (신라 -> [실라])
    3. **자소 분리 (Jamo Decomposition):** 음절을 초성, 중성, 종성으로 분해.
        - [실라] -> ㅅ, ㅣ, ㄹ, ㄹ, ㅏ
        - 도구: jamotools 또는 jamo 라이브러리.
    4. **거리 계산 (Levenshtein Distance):** 편집 거리 알고리즘 적용.
        - A를 B로 바꾸기 위해 몇 번의 삽입, 삭제, 교체가 필요한가?
        - 수식: $Score = 1 - (Distance / MaxLength)$
    5. **출력 (Output):** 0~100점 사이의 유사도 점수 (p_score)

2.2. [Track B] 관념 유사도 모델 (Conceptual Similarity Model)

"사전 구축 없이, 이미 학습된 거대 언어 모델의 뇌(Embedding)를 빌려 쓴다."

- **개발 철학:** 파인튜닝(Fine-tuning)이나 사전 DB 구축은 비효율적이다. 최신 임베딩 모델(SOTA)은 이미 다국어와 동의어 관계를 학습했다.
- **오해와 진실:**
    - (X) 표준국어대사전, 영어사전을 Vector DB에 넣어야 한다.
    - (O) 그냥 API에 두 단어를 던지면, AI가 알아서 의미 벡터를 뽑아준다.
- **작동 프로세스 (Process Flow):**
    1. **입력 (Input):** Text A ("Apple"), Text B ("사과")
    2. **임베딩 (Embedding):** OpenAI API (text-embedding-3-small 등) 호출.
        - 단어를 1536차원(혹은 그 이상)의 고차원 **벡터(숫자 배열)**로 변환.
        - Apple -> [0.01, 0.95, ...] (과일, 붉은색, IT 등의 속성값)
        - 사과 -> [0.02, 0.94, ...] (유사한 속성값)
    3. **유사도 계산 (Cosine Similarity):** 두 벡터 사이의 각도(Cos $₩theta$) 계산.
        - 두 화살표가 같은 방향을 가리킬수록 1(100%)에 가까움.

        - 도구: Python scikit-learn의 cosine_similarity 함수.

    4. **출력 (Output):** 0~100점 사이의 유사도 점수 (c_score)

2.3. [Track C] 시각 유사도 모델 (Visual Similarity Model) - Next Focus

"유일하게 데이터 학습이 필요한, 이 프로젝트의 최종 보스."

- **현황:** 아직 구체화 전 단계. 범용 API(Google Vision)는 객체 인식(사과, 컵)은 잘 하지만, **"로고의 표절 여부"**를 판단하는 능력은 부족함.
- **개발 방향: Siamese Network (샴 네트워크)** 도입 예정.
    - 두 개의 이미지를 동시에 넣고, 두 이미지의 '특징 거리(Feature Distance)'를 학습시키는 방식.
    - **필요 데이터:** Positive Pair(진품-짝퉁) / Negative Pair(진품-타인상표) 데이터셋 구축 필요.
- **비고:** 이 부분은 향후 논의를 통해 구체적인 모델 아키텍처와 데이터 생성 전략(GAN, Diffusion 활용 등)을 수립해야 함.

---

3. 최종 판단 엔진: RAG 기반 법리 해석기 (The Judge)

이 부분이 창훈 님이 가장 헷갈려 했던 부분이자, 우리 시스템의 **'화룡점정'**입니다.

3.1. RAG의 위치와 역할

- **위치:** 모든 모델(Track A, B, C)의 계산이 끝난 **가장 마지막 단계**.
- **역할:** Score(숫자)를 Legal Opinion(법적 의견)으로 번역하는 **통역사**.
    - 모델 내부에서 유사도를 판단하는 데 쓰이는 것이 아님.
    - 변리사가 고객에게 제출할 **"그럴싸한 근거"**를 만들어주는 생성형 AI 파트.

3.2. 작동 메커니즘 (Step-by-Step)

1. **Score Aggregation (점수 취합):**
    - p_score (호칭): 10점
    - c_score (관념): 98점
    - v_score (외관): 20점

2. **Rule-based Translation (규칙 기반 번역 - Python):**
    - 개발자가 정한 임계값(Threshold)에 따라 상태(Status) 문자열 생성.
    - if c_score > 80: concept_status = "매우 유사(High)"
    - Generated Context: "본 건은 호칭은 다르나, 관념 유사도가 매우 높음."
3. **Retrieval (법리 검색 - Vector Search):**
    - Query: "관념 유사 침해 판단 기준"
    - Result: "상표법 심사기준 제X조... 호칭이 달라도 관념이 동일하여 오인 혼동 우려가 있으면 유사하다."
4. **Generation (최종 추론 - LLM):**
    - Prompt: Context + Result + "변리사처럼 판단해줘."
    - **Final Output:** "두 상표는 외관 및 호칭의 차이에도 불구하고, '사과'와 'Apple'이라는 동일한 관념을 공유하고 있습니다. 심사기준 제X조에 의거, 이는 수요자에게 출처의 오인·혼동을 일으킬 우려가 있어 상표권 침해의 소지가 다분합니다."

---

4. 인프라 및 기술 스택 (Tech Stack) - Cost Efficiency 중심

초기 스타트업/토이 프로젝트 규모에 맞춰, **'직접 구축(Self-hosted)'**보다는 **'API 활용 (Managed Service)'** 위주로 재편했습니다.

4.1. Backend

- **Language:** Python 3.10+
- **Framework:** FastAPI (비동기 처리에 강함, AI 모델 서빙에 최적)

4.2. AI & Data Processing

- **Phonetic:** jamotools, ko-g2p, python-Levenshtein
- **Concept:** OpenAI API (text-embedding-3-small), scikit-learn
- **RAG Core:**
    - **LLM:** GPT-4o mini 또는 Gemini 1.5 Flash (가성비 추론 모델)
    - **Vector DB:** Pinecone (Free Tier 활용) 또는 ChromaDB (로컬) - *심사기준*

및 판례 저장용

4.3. Data Collection

- **Libraries:** BeautifulSoup4 (정적 파싱), Selenium (동적 크롤링 - 제한적 사용)
- **Target:** 공공데이터포털(Base), 네이버 지도(Target)

---

5. 향후 로드맵 (Next Steps)

이제 **텍스트(호칭/관념)** 모델은 설계가 끝났습니다. 코딩만 하면 됩니다. 남은 것은 **'이미지(Vision)'**입니다.

1. **Phase 1 (Done):** 전체 아키텍처 및 텍스트 모델 설계 확정.
2. **Phase 2 (Next):**
    - **Vision 모델 구체화:** Siamese Network 원리 학습.
    - **이미지 데이터셋 전략:** 짝퉁 로고 이미지를 어떻게 1,000장 확보할 것인가? (생성형 AI 활용 등 논의).
3. **Phase 3:** MVP 개발 착수 (통합 파이프라인 구축).